

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sn

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn import metrics

np.random.seed(42)

n = 500

x1 = np.random.uniform(-5, 5, n)
x2 = np.random.uniform(-2, 12, n)

# Quadratic boundary
boundary = 0.4 * (x1 ** 2) + 1

# Create classes
label = (x2 > boundary).astype(int)

data = pd.DataFrame({
    'x1': x1,
    'x2': x2,
    'label': label
})

print(data.shape)
print(data.head())

(500, 3)
   x1      x2  label
0 -1.254599  7.774264    1
1  4.507143  5.505349    0
2  2.319939  2.333387    0
3  0.986585  9.393130    1
4 -3.439814  7.586236    1

data.to_csv('svm_quadratic_500.csv', index=False)

print("CSV file created successfully!")

CSV file created successfully!

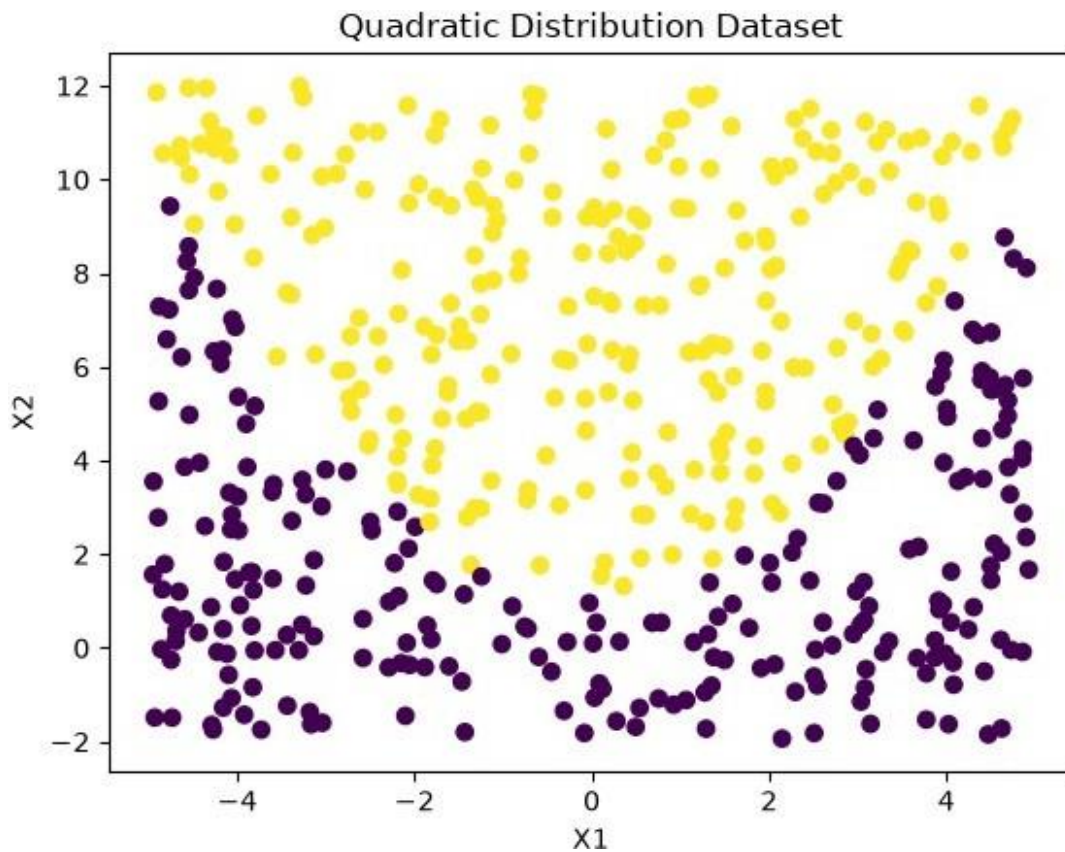
data.isnull().sum()

x1      0
x2      0
```

```
label      0
dtype: int64

plt.scatter(
    data['x1'],
    data['x2'],
    c=data['label']
)

plt.xlabel('X1')
plt.ylabel('X2')
plt.title('Quadratic Distribution Dataset')
plt.show()
```



```
x = data[['x1', 'x2']]
y = data['label']

x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.3,
    random_state=5
)
```

```

print(x_train.shape)
print(y_train.shape)
print(x_test.shape)
print(y_test.shape)

(350, 2)
(350,)
(150, 2)
(150,)

sc = StandardScaler()

x_train = sc.fit_transform(x_train)
x_test = sc.transform(x_test)

model = SVC(kernel='rbf', random_state=0)

model.fit(x_train, y_train)

svc_prediction = model.predict(x_test)

print("SVC Prediction:")
print(svc_prediction)

SVC Prediction:
[0 1 1 1 1 1 0 1 1 0 0 0 1 1 0 1 1 1 1 1 1 1 1 1 0 1 0 0 1 1 1 1 1
 0 1
 1 0 1 1 0 1 0 1 1 1 0 0 0 0 1 0 0 1 0 1 0 0 1 1 1 1 1 0 0 0 0 0 1 1 1
 1 0
 0 0 0 0 1 1 1 0 0 0 1 0 1 1 0 0 0 0 0 1 0 1 0 0 0 0 0 0 1 1 0 1 1 1 1
 1 0
 1 0 1 0 1 1 1 1 0 0 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0 1 1 0 1 1 1 1 0 0 1 1
 1 0
 1 1]

conf_mat = metrics.confusion_matrix(y_test, svc_prediction)

print("SVC [kernel - rbf]")
print("Confusion Matrix:")
print(conf_mat)

Accuracy_score = metrics.accuracy_score(
    y_test,
    svc_prediction
)

print("Accuracy Score:", Accuracy_score)

print(
    "Accuracy in Percentage:",

```

```

    int(Accuracy_score * 100),
    "%"
)

print(metrics.classification_report(
    y_test,
    svc_prediction
))

SVC [kernel - rbf]
Confusion Matrix:
[[68 11]
 [ 1 70]]
Accuracy Score: 0.92
Accuracy in Percentage: 92 %

```

	precision	recall	f1-score	support
0	0.99	0.86	0.92	79
1	0.86	0.99	0.92	71
accuracy			0.92	150
macro avg	0.92	0.92	0.92	150
weighted avg	0.93	0.92	0.92	150

```

conf_mat = pd.crosstab(
    y_test,
    svc_prediction,
    rownames=['Actual'],
    colnames=['Predicted']
)

sn.heatmap(conf_mat, annot=True)

plt.title('SVM - Quadratic Dataset')
plt.show()

```

